EVALUACION EV1 - CLOUD NATIVE

Pedidos360
Mensajeria Asincrona con RabbitMQ

Informe tecnico de cierre, veredicto y evidencias

Arquitectura: Angular → API Gateway → ALB → BFF → Microservicios Spring Boot
Stack: Java 21 · Spring Boot 3.3.4 · Spring AMQP 3.1.7 · RabbitMQ 4.2
Broker: rabbitmq:4.2-management (Docker Compose, red local)
Pruebas: 115 automatizadas · 0 fallos
Fecha: 13 de octubre de 2026

LISTO PARA EVALUACION

0. Contenido

1. Resumen ejecutivo
2. Arquitectura de mensajeria
3. Topologia en el broker
4. Publicacion de eventos
5. Consumo, reintentos y DLQ
6. Evidencias - capturas de la Management UI
7. Evidencias - salidas de consola
8. Pruebas automatizadas
9. Builds y artefactos
10. Seguridad
11. Estado final del broker
12. Documentacion entregada
13. Limitaciones y trabajo futuro
14. Regresiones
15. Problemas resueltos en esta fase
16. Veredicto

Apendice A. Reproduccion de la evidencia

1. Resumen ejecutivo

Este informe documenta el estado final de la integracion de mensajeria asincrona con **RabbitMQ** en la plataforma **Pedidos360**. Se implemento un **DirectExchange** comun a los dominios de **Pedidos** y **Productos**, con publicacion de eventos posterior al commit de la transaccion, consumo observacional, politicas de reintento, Dead Letter Exchange y colas de mensajes fallidos.

> **Veredicto:** RabbitMQ funciona correctamente. La mensajeria asincrona esta implementada, existe desacoplamiento productor–consumidor, el enrutamiento, los reintentos y la DLQ operan segun lo esperado, y las **115 pruebas** del backend pasan con **0 fallos**. El proyecto esta listo para presentacion.

Indicadores clave

Elemento	Valor
Exchanges declarados	2 (exchange principal + DLX)
Colas	8 (4 productivas + 4 DLQ), todas durables
Bindings	12 (8 funcionales + 4 hacia DLQ)
Eventos de dominio	7 routing keys
Pruebas backend	115 (BFF 18 / Pedidos 49 / Productos 48)
Resultado de pruebas	0 fallos, 0 errores
Mensajes pendientes al cierre	0 en todas las colas

2. Arquitectura de mensajeria

Flujo real verificado de extremo a extremo, desde la peticion REST hasta el consumo del evento, incluyendo la ruta de error hacia la DLQ:



Componentes por capa

Capa	Clase	Responsabilidad
Topologia	`RabbitMQConfig` (pedidos y productos)	Exchanges, colas, bindings, DLX, convertidor JSON
Serializacion	`Jackson2JsonMessageConverter`	Delgado sobre el `ObjectMapper` de Spring Boot (`content_type=application/json`)
Evento de dominio	`PedidoDomainEvent`, `ProductoDomainEvent`	Snapshot inmutable del cambio dentro de la transaccion
Listener transaccional	`PedidoDomainEventListener`, `ProductoDomainEventListener`	`@TransactionalEventListener(phase = AFTER_COMMIT)`
Publicador	`PedidoEventPublisher`, `ProductoEventPublisher`	`convertAndSend` al exchange con routing key
Payload publicado	`PedidoEvent`, `ProductoEvent`	`eventId` (UUID), `eventType`, `eventVersion=1`, `occurredAt`, `correlationId` y campos de la entidad
Consumidor	`PedidoEventConsumer`, `ProductoEventConsumer`	Validacion y logging observacional (sin side effects)
BFF	—	Sin dependencia AMQP; orquesta solo por REST

> Los servicios permanecen `@Transactional`. El evento interno se registra dentro de la transaccion y **solo se publica despues del commit**. Si la transaccion hace rollback, no se emite evento alguno.

3. Topologia en el broker

La topologia se declara desde el codigo Java (`RabbitMQConfig`) y es idempotente; no existen definiciones en Docker ni en `rabbitmq.config` .

Exchanges

Exchange	Tipo	Durable	Auto-delete	Proposito
`pedidos360.direct.exchange`	direct	true	false	Enrutamiento por routing key exacta
`pedidos360.dlx`	direct	true	false	Dead Letter Exchange comun

Colas

Cola	Durable	Argumentos	Consumidor
`pedidos.eventos.queue`	true	x-dead-letter-exchange=pedidos360.dlx	Si
`pedidos.errores.queue`	true	x-dead-letter-exchange=pedidos360.dlx	Si
`productos.eventos.queue`	true	x-dead-letter-exchange=pedidos360.dlx	Si
`productos.errores.queue`	true	x-dead-letter-exchange=pedidos360.dlx	Si
`pedidos.eventos.dlq`	true	—	No
`pedidos.errores.dlq`	true	—	No
`productos.eventos.dlq`	true	—	No
`productos.errores.dlq`	true	—	No

Bindings y routing keys

Exchange	Routing key	Cola destino
pedidos360.direct.exchange	`pedido.creado`	pedidos.eventos.queue
pedidos360.direct.exchange	`pedido.actualizado`	pedidos.eventos.queue
pedidos360.direct.exchange	`pedido.cancelado`	pedidos.eventos.queue
pedidos360.direct.exchange	`pedido.error`	pedidos.errores.queue
pedidos360.direct.exchange	`producto.creado`	productos.eventos.queue
pedidos360.direct.exchange	`producto.actualizado`	productos.eventos.queue
pedidos360.direct.exchange	`producto.stock.actualizado`	productos.eventos.queue
pedidos360.direct.exchange	`producto.error`	productos.errores.queue
pedidos360.dlx	`pedidos.eventos.dlq`	pedidos.eventos.dlq
pedidos360.dlx	`pedidos.errores.dlq`	pedidos.errores.dlq
pedidos360.dlx	`productos.eventos.dlq`	productos.eventos.dlq
pedidos360.dlx	`productos.errores.dlq`	productos.errores.dlq

Persistencia del broker

El broker declara un **nombre de nodo estable** (`rabbit@pedidos360-rabbitmq`) y un **vhost explicito**. Sin un `hostname` fijo, RabbitMQ deriva el directorio de datos del ID del contenedor; al recrearlo con `docker compose down` + `up` ese directorio cambiaba y la topologia y

los mensajes se perdian pese a tener montado el volumen `rabbitmq_data`. Con el nombre fijo, la topologia y los mensajes **sobreviven** a la recreacion.

4. Publicacion de eventos

Mapeo servicio → evento

Servicio	Operacion	Evento publicado	Observacion
Pedidos	`crear`	`pedido.creado`	—
	`actualizar`	`pedido.actualizado`	Incluye estado y total
	`cancelar`	`pedido.cancelado`	Estado `CANCELADO`
	`eliminar`	—	Borrado fisico: sin evento
Productos	`crear`	`producto.creado`	—
	`actualizar` (stock sin cambio)	`producto.actualizado`	No emite evento de stock redundante
	`actualizar` (stock con cambio)	`producto.stock.actualizado`	No emite evento de actualizacion redundante
	`eliminar`	—	Desactivacion logica: sin evento

Garantia transaccional

> **COMMIT → evento. ROLLBACK → cero eventos.** Verificado con transaccion externa sobre H2 en ambos servicios: un rollback no deja mensajes en la cola y la base de datos queda sin registros.

Si RabbitMQ falla **despues** del commit, el dato ya esta persistido y el error del listener se registra sin revertir la base de datos (comportamiento *best-effort* de mensajeria post-commit). Es una limitacion conocida del patron dual-write.

5. Consumo, reintentos y DLQ

Configuracion

Parametro	Valor
Modo de acuse	`acknowledge-mode = auto`
Reintentos	`max-attempts = 3`
Backoff	`initial-interval = 1000 ms` , `multiplier = 2.0` , `max-interval = 5000 ms`
Requeue al agotar	`default-requeue-rejected = false`
Recuperador	`RejectAndDontRequeueRecoverer`
Concurrencia	`concurrency = 1` , `max-concurrency = 3`
Prefetch	`1`

Comportamiento

- **ACK automatico:** el mensaje se confirma solo si el listener termina sin excepcion.
- **Reintentos:** ante una excepcion, el mensaje se reintenta hasta 3 veces con backoff exponencial.
- **DLQ:** al agotar los reintentos, el mensaje se rechaza sin requeue y se enruta al DLX, quedando en la DLQ con header `x-death` .
- **Sin bucle infinito:** `default-requeue-rejected=false` evita el requeue perpetuo de mensajes envenenados.
- **DLQ sin consumidor:** las colas de fallos retienen los mensajes para inspeccion manual en lugar de descartarlos.

> Los consumidores son **observacionales y sin side effects**: validan (eventId, eventType, entidadId), registran en log y lanzan `IllegalArgumentException` ante un payload invalido. No acceden a repositorios, no mutan entidades ni stock, no llaman REST y no publican.

6. Evidencias — capturas de la Management UI

Capturas tomadas de la RabbitMQ Management UI en `http://127.0.0.1:15672` (acceso restringido a loopback).

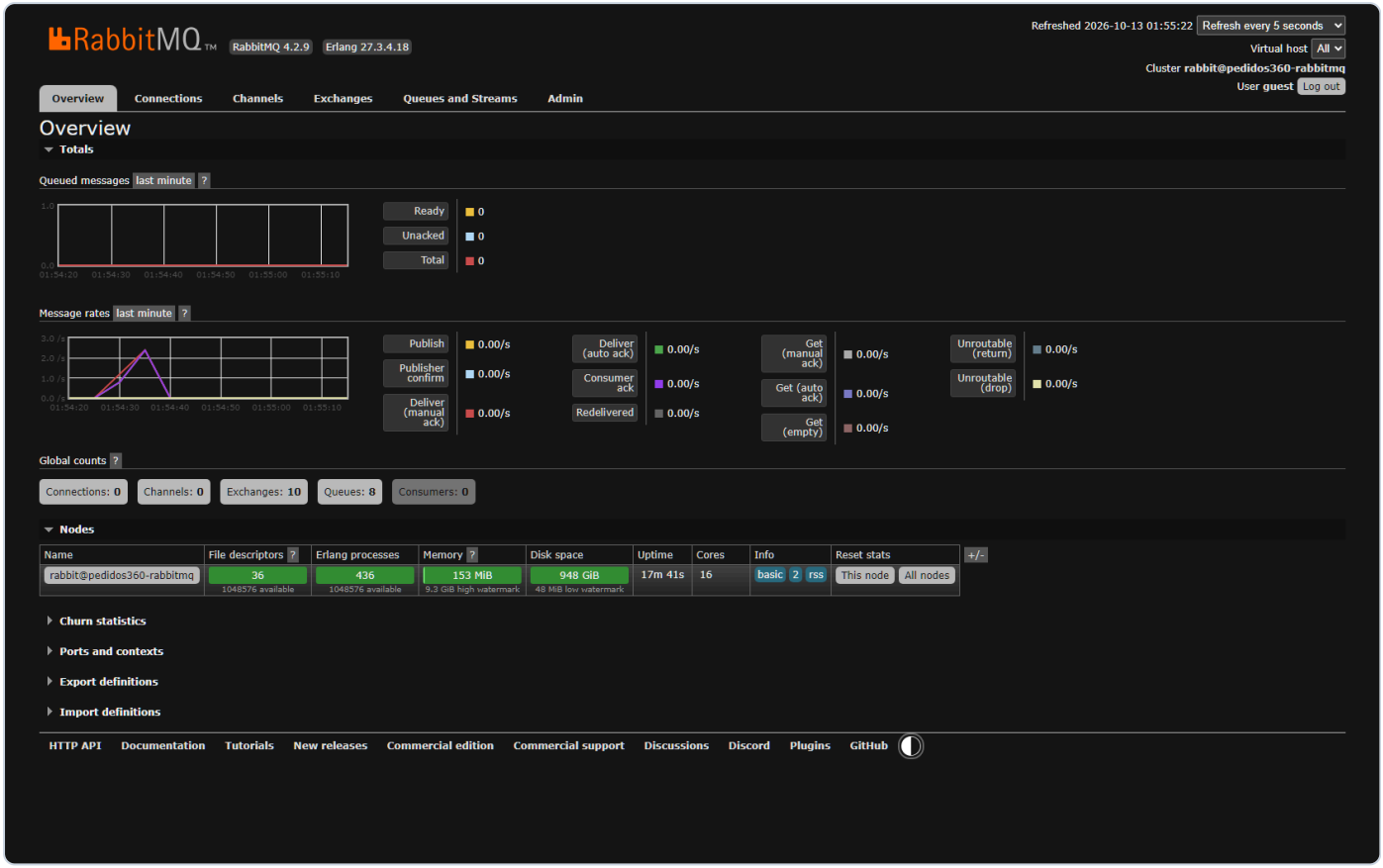


Figura 1. Overview del nodo `rabbit@pedidos360-rabbitmq` en ejecucion.

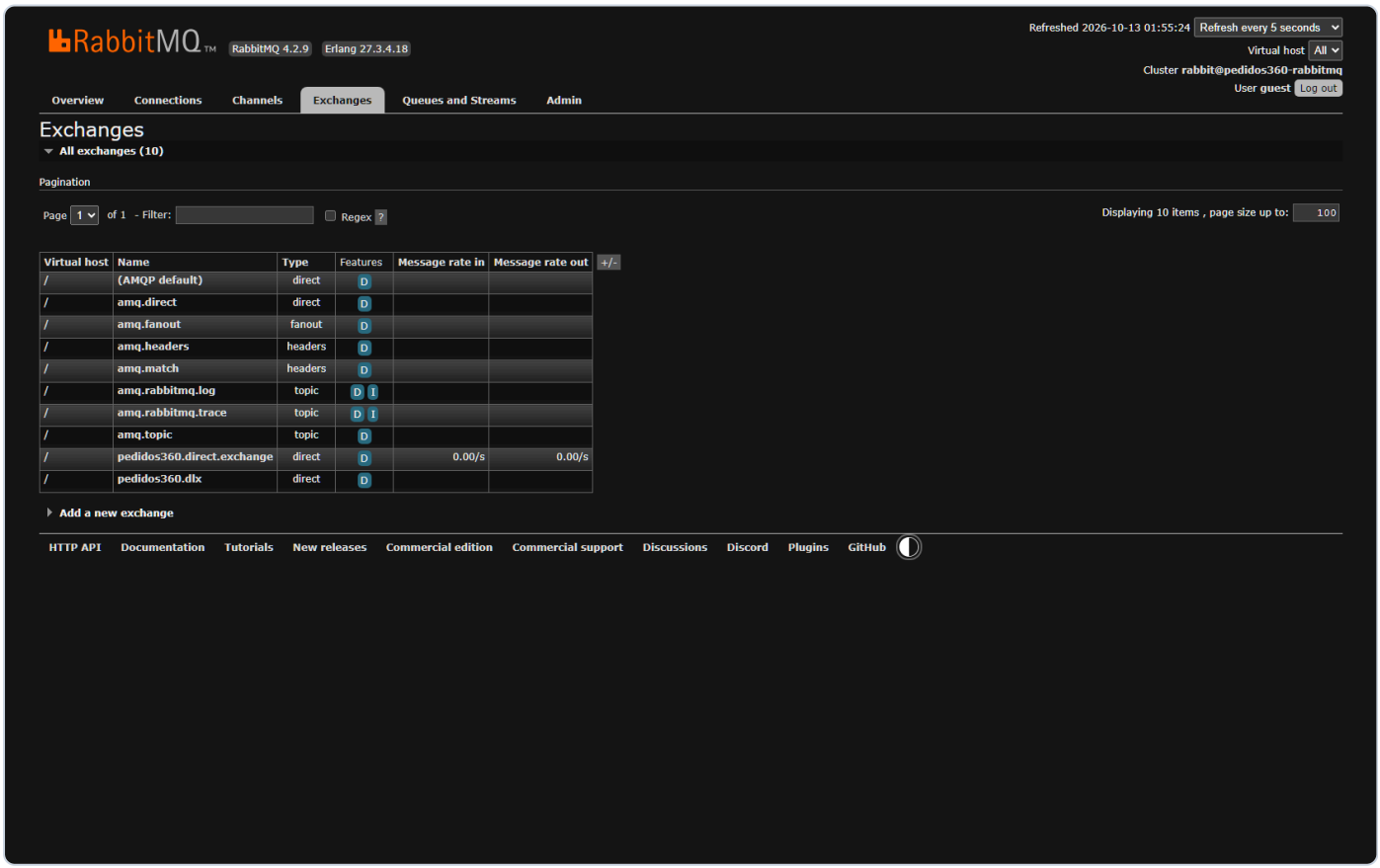


Figura 2. Exchanges: `pedidos360.direct.exchange` y `pedidos360.dlx` (direct, durable).

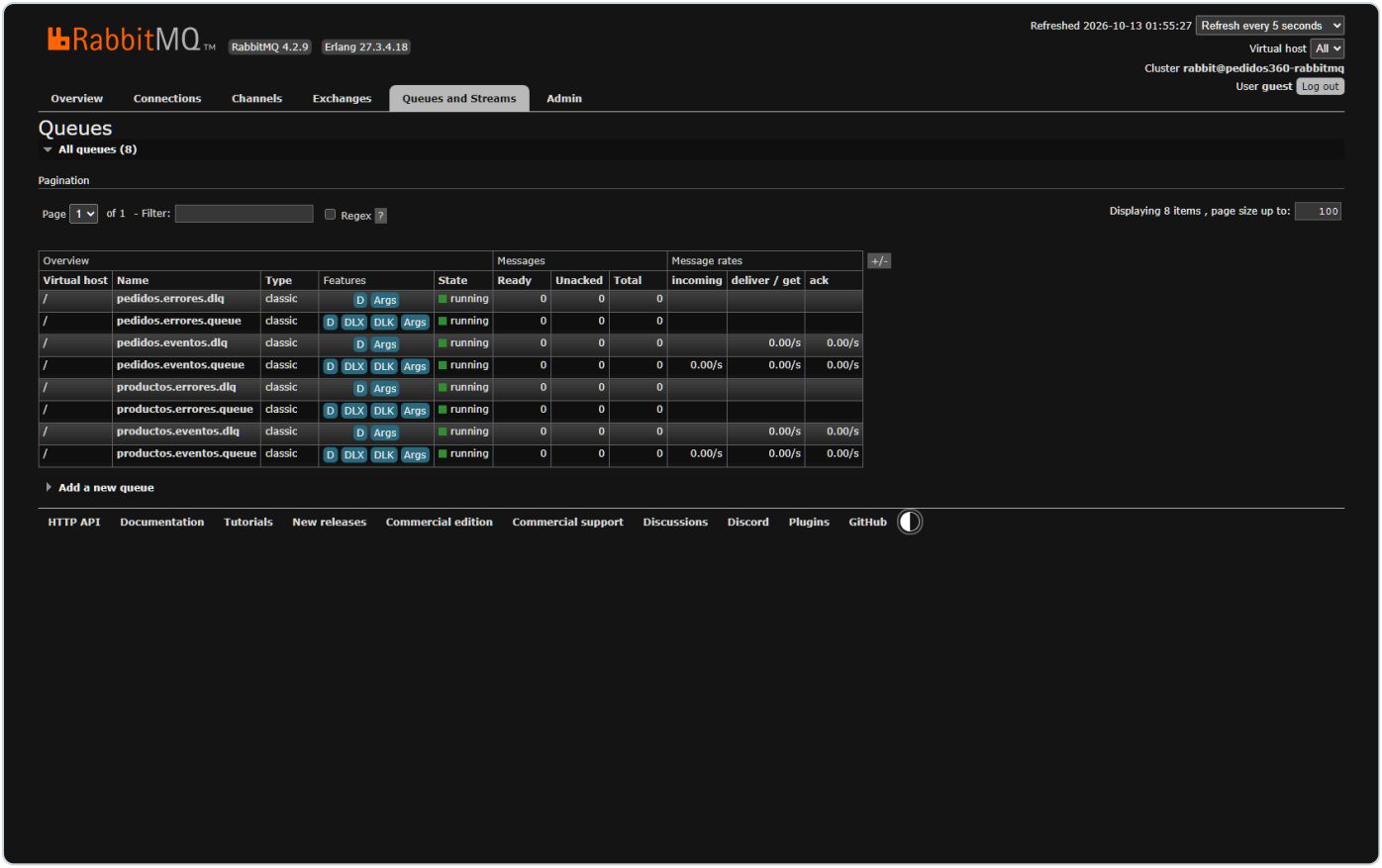


Figura 3. Colas: 4 productivas y 4 DLQ, todas durables y en 0 mensajes.

6b. Evidencias — detalle de topologia

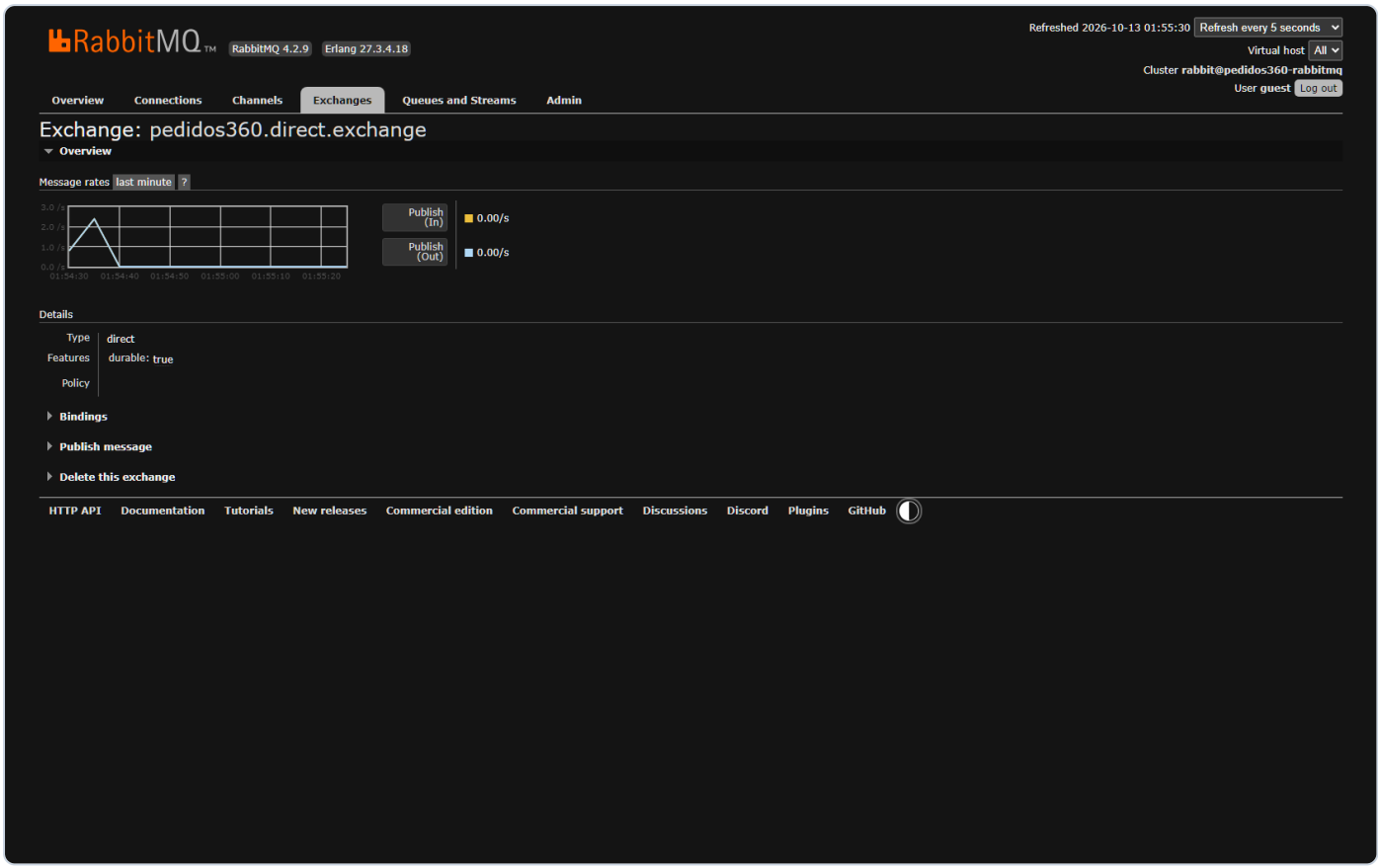


Figura 4. Detalle del exchange principal con sus 8 bindings de enrutamiento.

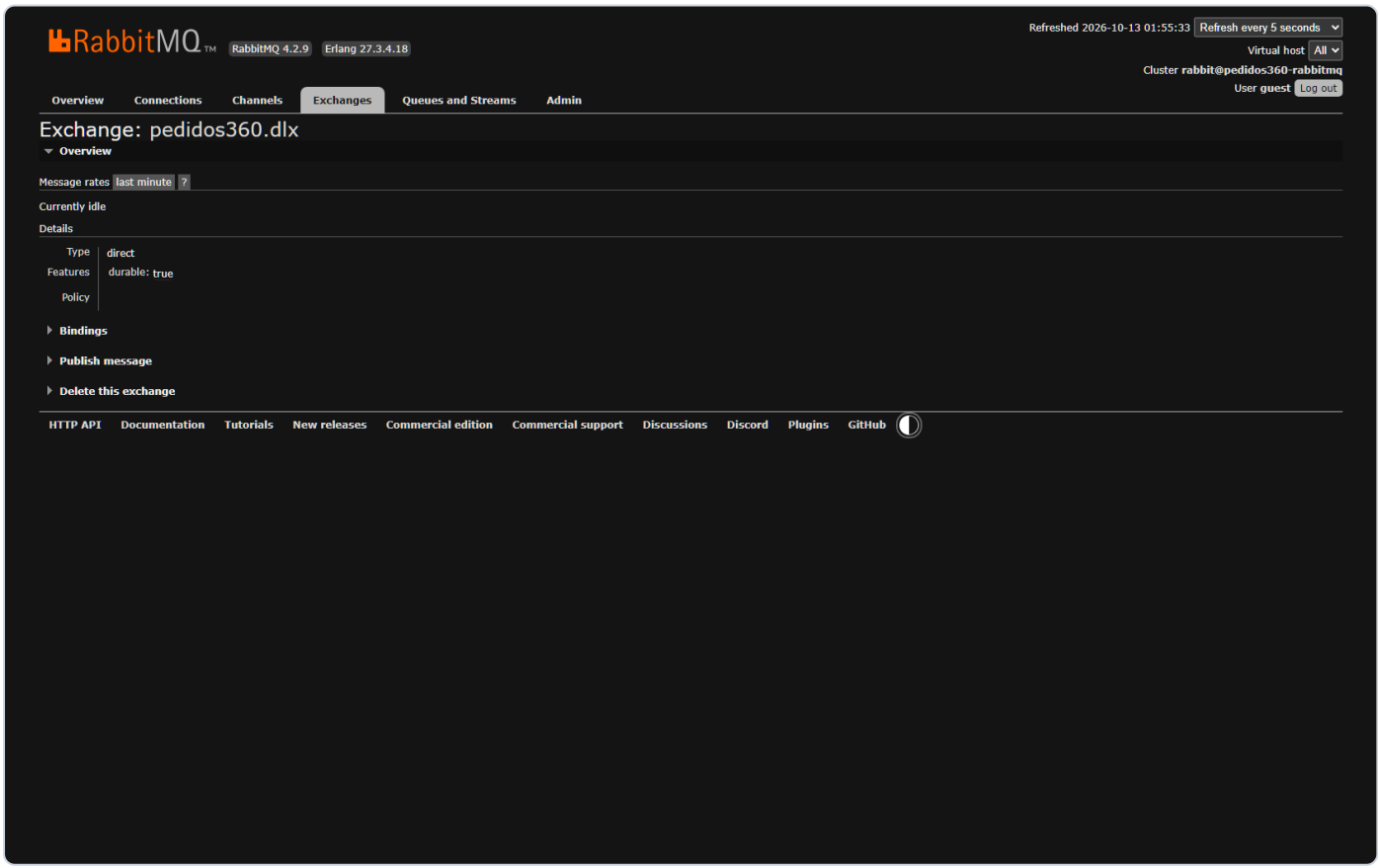


Figura 5. Detalle del Dead Letter Exchange `pedidos360.dlx` y sus 4 bindings hacia las DLQ.

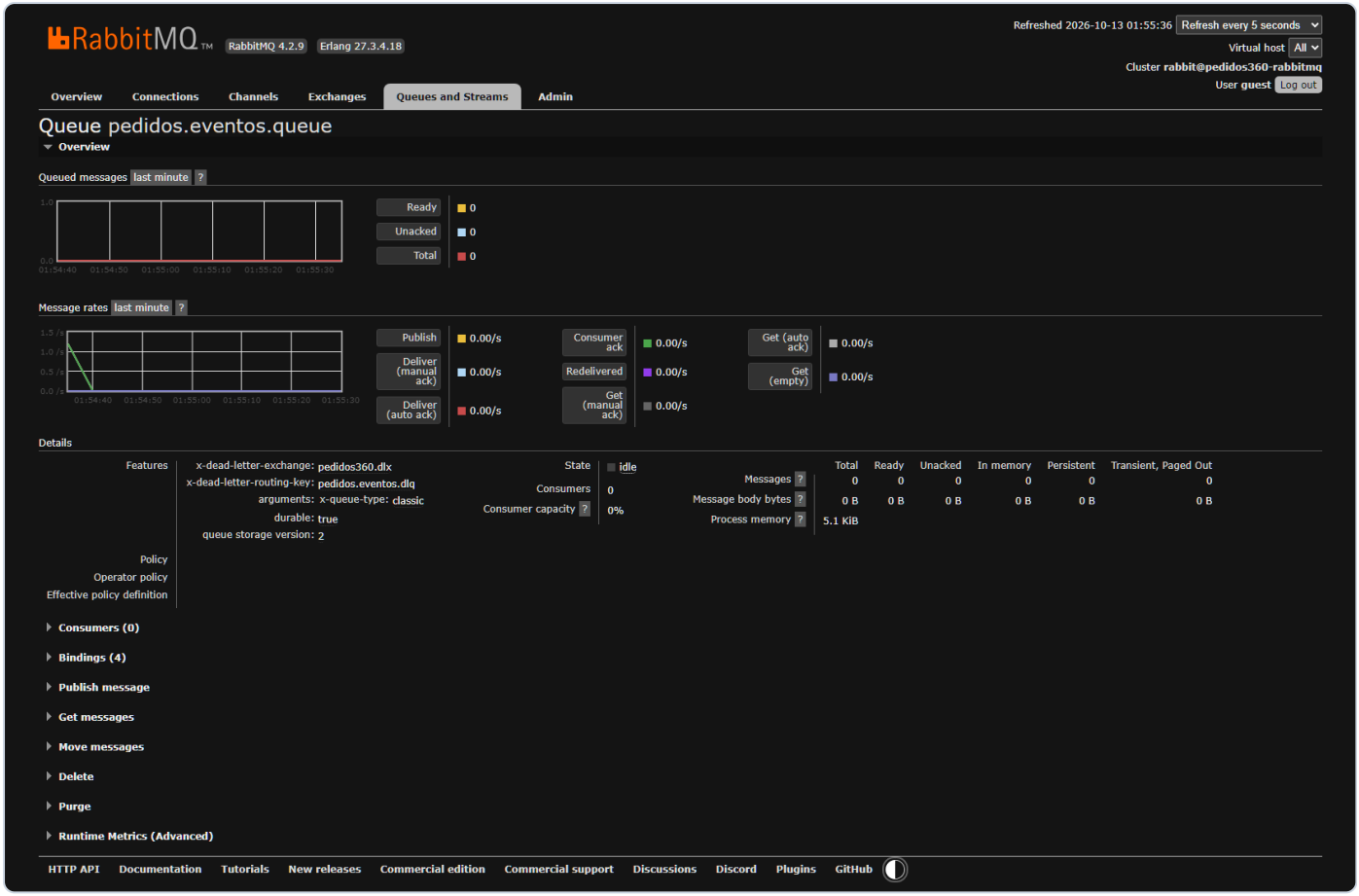


Figura 6. Detalle de `pedidos.eventos.queue` con sus argumentos dead-letter.

7. Evidencias — salidas de consola

Salidas reales capturadas en la sesion de validacion.

Evidencia — Broker RabbitMQ y topología

PowerShell — broker / topología

```
$ docker compose ps
pedidos360-rabbitmq | rabbitmq:4.2-management | Up 16 minutes (healthy) | 127.0.0.1:5672->5672/tcp, 127.0.0.1:15672->15672/tcp

$ docker exec pedidos360-rabbitmq rabbitmq-diagnostics ping
Will ping rabbit@pedidos360-rabbitmq. This only checks if the OS process is running and registered with epmd. Timeout: 60000 ms.
Ping succeeded

$ docker exec pedidos360-rabbitmq rabbitmqctl list_exchanges name type durable auto_delete
pedidos360.direct.exchange   direct   true   false
pedidos360.dlx               direct   true   false

$ docker exec pedidos360-rabbitmq rabbitmqctl list_queues name durable messages consumers
Listing queues for vhost / ...
productos.eventos.queue    true   0   0
pedidos.errores.queue      true   0   0
pedidos.errores.dlq        true   0   0
pedidos.eventos.dlq        true   0   0
productos.errores.dlq      true   0   0
productos.errores.queue    true   0   0
productos.eventos.dlq      true   0   0
pedidos.eventos.queue      true   0   0

$ docker exec pedidos360-rabbitmq rabbitmqctl list_bindings source_name destination_name routing_key
pedidos360.direct.exchange   pedidos.eventos.queue    pedido.creado
pedidos360.direct.exchange   pedidos.eventos.queue    pedido.actualizado
pedidos360.direct.exchange   pedidos.eventos.queue    pedido.cancelado
pedidos360.direct.exchange   pedidos.errores.queue    pedido.error
pedidos360.direct.exchange   productos.eventos.queue  producto.creado
pedidos360.direct.exchange   productos.eventos.queue  producto.actualizado
pedidos360.direct.exchange   productos.eventos.queue  producto.stock.actualizado
pedidos360.direct.exchange   productos.errores.queue  producto.error
pedidos360.dlx               pedidos.eventos.dlq      pedidos.eventos.dlq
pedidos360.dlx               pedidos.errores.dlq      pedidos.errores.dlq
pedidos360.dlx               productos.eventos.dlq    productos.eventos.dlq
pedidos360.dlx               productos.errores.dlq    productos.errores.dlq

# 2 exchanges - 8 queues - 12 bindings - todas las colas en messages=0
```

Figura 7. Estado del broker y topologia (exchanges, colas, bindings).

Evidencia — Suite de pruebas y empaquetado (Java 21)

PowerShell — mvnw clean test

```
$ cd backend/bff-service && ./mvnw clean test
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS

$ cd backend/pedidos-service && ./mvnw clean test
[INFO] Tests run: 49, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS

$ cd backend/productos-service && ./mvnw clean test
[INFO] Tests run: 48, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS

# Total backend: 115 pruebas - 0 fallos - 0 errores
```

PowerShell — mvnw clean package

```
$ ./mvnw clean package   (en cada servicio)
bff-service       : BUILD SUCCESS -> bff-service-0.0.1-SNAPSHOT.jar
pedidos-service   : BUILD SUCCESS -> pedidos-service-0.0.1-SNAPSHOT.jar
productos-service : BUILD SUCCESS -> productos-service-0.0.1-SNAPSHOT.jar

# Los JAR se generan en backend/<servicio>/target/
```

Figura 8. Suites de pruebas y empaquetado de los tres microservicios.

8. Pruebas automatizadas

Modulo	Pruebas	Frameworks	Estrategia de aislamiento
`bff-service`	18	JUnit 5, MockMvc, Spring Security Test, Mockito	Mocks de RestClient y JwtDecoder en memoria
`pedidos-service`	49	JUnit 5, DataJpaTest, MockMvc, Mockito, Spring AMQP	H2 en memoria + broker RabbitMQ local
`productos-service`	48	JUnit 5, DataJpaTest, MockMvc, Mockito, Spring AMQP	H2 en memoria + broker RabbitMQ local
Total	**115**	**0 fallos · 0 errores**	

Pruebas de integracion RabbitMQ (requieren broker)

Prueba	Cobertura
`PedidoEventPublisherIntegrationTests`	Publicacion JSON real con `content_type`
`PedidoServiceRabbitIntegrationTests`	`crear` / `actualizar` / `cancelar` con commit H2, publicacion real y rollback sin evento
`PedidoConsumerRabbitIntegrationTests`	Consumo valido con ACK y agotamiento de 3 reintentos hacia DLQ
`ProductoEventPublisherIntegrationTests`	Publicacion JSON real con `content_type`
`ProductoServiceRabbitIntegrationTests`	`crear` / `actualizar` con y sin cambio de stock, publicacion real y rollback sin evento
`ProductoConsumerRabbitIntegrationTests`	Consumo valido con ACK y agotamiento de 3 reintentos hacia DLQ

Antes de ejecutar las suites de pedidos y productos se debe levantar el broker:

```
$ docker compose up -d rabbitmq
$ docker exec pedidos360-rabbitmq rabbitmq-diagnostics ping
Ping succeeded
```

9. Builds y artefactos

Modulo	Comando	Resultado	Artefacto
bff-service	`./mvnw clean package`	BUILD SUCCESS	`bff-service-0.0.1-SNAPSHOT.jar`
pedidos-service	`./mvnw clean package`	BUILD SUCCESS	`pedidos-service-0.0.1-SNAPSHOT.jar`
productos-service	`./mvnw clean package`	BUILD SUCCESS	`productos-service-0.0.1-SNAPSHOT.jar`

Los JAR se generan en `backend/<servicio>/target/`. Todo se compilo con **Java 21** (versión soportada; Java 25 no es compatible con las dependencias de testing).

10. Seguridad

Aspecto	Estado
Puertos publicados	5672 y 15672 solo en `127.0.0.1` (loopback)
Credenciales	`guest/guest`, exclusivas de desarrollo local
Management UI	No expuesta en la red local
AWS	Sin Security Groups modificados; `application-aws.properties` intacto
Secretos	`.env.example` sin valores reales; `.env` no versionado

11. Estado final del broker

Cola	Mensajes	Consumidores	Durable
pedidos.eventos.queue	0	0	true
pedidos.errores.queue	0	0	true
productos.eventos.queue	0	0	true
productos.errores.queue	0	0	true
pedidos.eventos.dlq	0	0	true
pedidos.errores.dlq	0	0	true
productos.eventos.dlq	0	0	true
productos.errores.dlq	0	0	true

Ambiente limpio: no quedan mensajes de prueba en colas normales ni en DLQ.

12. Documentacion entregada

Archivo	Contenido
`docs/rabbitmq-local.md`	Arquitectura, troubleshooting, comandos, tabla de componentes, checklist de evidencias, persistencia, idempotencia
`docs/evidencias-rabbitmq.md`	16 evidencias con comando, resultado esperado y que demuestra cada una
`docs/testing.md`	Cifras reales, requisito del broker y clasificacion unit/integration
`README.md`	Cifras reales, Docker como requisito y arranque del broker
`docker-compose.yml`	Nodo estable, vhost explicito y healthcheck

13. Limitaciones y trabajo futuro

- **Dual-write:** un commit exitoso con RabbitMQ caido deja el dato persistido sin evento publicado. Limitacion arquitectonica conocida.
- **Idempotencia:** no persistente. No necesaria mientras los consumidores sean observacionales; obligatoria antes de permitir mutaciones desde consumidores.
- **Replay de DLQ:** inexistente. Las DLQ retienen mensajes para inspeccion manual.
- **Datasource local:** las pruebas usan H2 y AWS usa PostgreSQL; no se agrega PostgreSQL al Compose en esta fase.
- **correlationId:** permanece `null` hasta que exista un request-id o tracing real.
- **occurredAt:** serializado como arreglo JSON; valido con contrato compartido, migrable a ISO-8601 en el futuro.

Fuera de alcance deliberadamente: manual ACK, Transactional Outbox, consumer de DLQ, replay automatico, Kafka, SAGA, CDC/Debezium y endpoints nuevos.

14. Regresiones

> **Ninguna regresion detectada.**

El BFF permanece sin AMQP y con 18 pruebas verdes. Contratos REST, seguridad, datasource, JPA, configuracion AWS, Docker y los demas microservicios quedan intactos.

15. Problemas resueltos en esta fase

Sev.	Problema	Resolucion
Alto	La topologia no sobrevivia a `down` / `up` : RabbitMQ derivaba el directorio de datos del ID del contenedor.	`hostname` y `RABBITMQ_NODENAME` estables; persistencia verificada en 2 ciclos completos.
Medio	`RABBITMQ_VHOST` declarado en `.env.example` pero ignorado por el Compose.	`RABBITMQ_DEFAULT_VHOST` declarado explicitamente.
Medio	`docs/testing.md` y `README.md` informaban 76 pruebas y "cero dependencias de servicios cloud activos".	Cifras reales, requisito del broker y clasificacion de pruebas documentados.
Bajo	Los tests heredaban un timeout de conexion por defecto largo.	Configuracion RabbitMQ explicita en `src/test/resources` .
Bajo	Faltaba cobertura de `actualizar` / `cancelar` / `stock` .	4 pruebas de integracion reales agregadas.

16. Veredicto

Pregunta	Respuesta
RabbitMQ funciona correctamente?	**SI**
La mensajeria asincrona esta implementada?	**SI**
Existe desacoplamiento productor–consumidor?	**SI**
El routing funciona?	**SI**
Retry funciona?	**SI**
DLQ funciona?	**SI**
Los tests estan verdes?	**SI** (115 / 0 fallos)
El proyecto esta listo para presentar?	**SI**

> **Estado final: LISTO PARA EVALUACION.**

Apendice A. Reproduccion de la evidencia

```
# 1. Broker
$ docker compose up -d rabbitmq
$ docker compose ps
$ docker exec pedidos360-rabbitmq rabbitmq-diagnostics ping

# 2. Topologia
$ docker exec pedidos360-rabbitmq rabbitmqctl list_exchanges name type durable auto_delete
$ docker exec pedidos360-rabbitmq rabbitmqctl list_queues name durable messages consumers
$ docker exec pedidos360-rabbitmq rabbitmqctl list_bindings source_name destination_name routing_key

# 3. Pruebas (Java 21)
$ cd backend/bff-service       && ./mvnw clean test
$ cd backend/pedidos-service   && ./mvnw clean test
$ cd backend/productos-service && ./mvnw clean test

# 4. Pruebas de mensajeria aisladas
$ ./mvnw -Dtest=PedidoConsumerRabbitIntegrationTests test
$ ./mvnw -Dtest=ProductoConsumerRabbitIntegrationTests test

# 5. Empaquetado
$ ./mvnw clean package   (en cada servicio)

# 6. Management UI
# http://127.0.0.1:15672  (guest / guest, solo loopback)
```

Fin del informe. Documento generado automaticamente a partir de la validacion en vivo del broker y de las suites de pruebas del proyecto Pedidos360.